Modularity and Python Modules

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

After the refactoring we performed, `triage.py` is clean, but it is still **one file**. Today we turn it into a package of small modules, working in the same way.

Concepts covered: modularity (cohesion, coupling, interfaces, information hiding), Python modules and packages (imports, the `_` convention, `__name__`, `__init__.py`, `__main__.py`, the search path, the src layout), data classes and enumerations, the difference between a refactoring and a feature, and the five **SOLID** design principles.

If you fall behind

During the lecture, do not stop to catch up: follow the explanation, and complete the missing steps after the lecture.

You find every step in the branch **`lab-01-solution`** of the course repository: open `unica-isde/heron` on GitHub, choose the branch `lab-01-solution`, and click the list of commits. Each commit is one step, with its message and its full change. The appendix at the end of this deck shows the same changes.

Modularity

What is modularity?

Real software is written by teams, and no person can keep a large program in mind at once. We must divide a problem into simpler problems.

A system that can be divided into smaller parts is **modular**. A **module** is a part of the system that **can be considered separately**: you can understand it, change it and test it without studying all the rest.

In Python a module is a `.py` file, but the idea is general: a module can also be a class, a package or a whole service.

Why modularity helps (1/2)

- **manage complexity**: break a complex system into simpler parts (top-down), or build it from existing parts (bottom-up)
- **anticipate change**: you can identify the modules that will probably change, and keep the change inside them
- **change the system** by modifying a small set of parts. In a monolithic script, how much code must you master before a small change?
- **work in a team**: different people work on different modules at the same time

Why modularity helps (2/2)

- **separation of concerns**: one module per aspect of the problem, so you can focus on one at a time
- **understandable** software: you understand the system as a composition of its parts
- **isolate errors**: check one module at a time instead of the whole program
- **reuse**: a module can be used by another program; a piece of a huge script cannot

For `heron`: after refactoring, the indicators are separate functions, but they still live in the same file as the command-line code. Reusing the scoring in a web API (as we'll do later) would mean importing a script that also reads files and prints.

Cohesion

Cohesion is the degree to which the elements **inside** a module belong together. We want it **high**: instructions and data that cooperate for the same goal live in the same module, and nothing else does.

Our objective is thus to have separate modules. For instance, `cli.py` to parse the command line arguments; `parser.py` to read/parse the emails; `scorer.py` containing only what turns the points of the indicators into a verdict.

Low cohesion is the opposite: `triage.py` mixes file access, header parsing, scoring and printing in one place.

Coupling

Coupling is the degree to which modules **depend on each other**: how much one module must know about another to work. We want it **low**.

- two modules that share global variables are tightly coupled: a change in one silently changes the other (the old `scores` and `verdicts` globals)
- at the end of today, `heron`'s `cli.py` uses the rest of the code through two functions (the interface), and knows nothing about the indicators or their weights (the inner implementation)
- the other modules (`models`, `parser`, `features`, `scorer`) never import `cli.py`: the command line can be replaced without touching them

High cohesion and low coupling together keep a change local: you change one module, and the others do not notice.

Interfaces

The **interface** of a module is the set of services it offers to other modules (its **clients**): the functions, classes and constants they may use.

- the **implementation** (how the services work) is a **secret** of the module
- clients know the module only through its interface; if the interface does not change, the implementation can change without breaking any client
- a module can be used and tested as a **black box**, through its interface only

What should the interface show?

As little as possible: just enough for clients to use the services.

Every unnecessary detail in the interface:

- makes the interface more complex to learn and use
- makes the whole system harder to understand
- increases the chance that a change in the implementation forces a change in the clients

Abstraction, encapsulation, information hiding

- **Abstraction**: the interface is an abstraction of the module. Knowing the interface is *enough* to use the module: you do not *need* to know anything else.
- **Encapsulation**: the details, the implementation and the state of the module are hidden. A client *cannot* reach anything beyond the interface.
- **Information hiding** = abstraction + encapsulation.

Authors do not all agree on these definitions (see Stevens and Pooley, *Using UML*, 2006). In Python, encapsulation is a **convention**, not a rule the interpreter enforces: the next slides show how it works.

Modules in Python

A module is a file

In Python, every `.py` file is a **module**. Its name is the file name without `.py`. Modules can also be:

- written in C, usually for speed (for example `math`)
- **built-in**: compiled into the interpreter and always available (for example `sys`)

We write modules in Python: one `.py` file each.

Names (PEP 8): modules have short, all-lowercase names; underscores are allowed if they help readability (`features.py`, `scorer.py`). Packages have short, all-lowercase names, preferably without underscores (`heron`).

Importing a module

`import` gives access to another module's names, through the module's name. This avoids collisions: two modules can each define a function with the same name.

Statement	You then write
`import heron.scorer`	`heron.scorer.triage(email)`
`from heron import scorer`	`scorer.triage(email)`
`from heron.scorer import triage`	`triage(email)`
`from heron.scorer import triage as run_checks`	`run_checks(email)`

`from heron.scorer import *` imports every name that does not start with `_`. **Avoid it**: you cannot see where a name comes from, and an imported name can silently replace one of yours.

Importing runs the module

The first time a module is imported, Python **runs all its top-level code**: `def` statements define functions, assignments create constants, but a `print()` at top level would print too.

To run some code **only when the file is run as a script**, Python sets the special variable `__name__`: it is the module's name when the module is imported, and the string `"__main__"` when the file is run directly. The end of `triage.py`:

```
if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("usage: python triage.py <folder-with-eml-files>", file=sys.stderr)
        sys.exit(2)
```

So `import triage` defines the functions without running the triage; `python3 triage.py data/samples` runs it.

Packages

A **package** is a **folder** of modules. It groups related modules and gives them a common prefix with the dot notation: `heron.parser`, `heron.scorer`. Packages avoid collisions between module names, as modules avoid collisions between function names.

- a file `__init__.py` in the folder runs when the package (or any module in it) is first imported; it can define package-level names, such as the version of the package
- a file `__main__.py` makes the package runnable: `python3 -m heron` runs `heron/__main__.py`
- since Python 3.3 a folder without `__init__.py` is also a package (a *namespace package*), but a normal package should have one

Where Python looks for modules

`import heron` makes Python search for a module or package named `heron` in the folders of the list `sys.path`, in order:

1. the folder of the script being run (or the current folder, for `python -m`)
2. the folders listed in the environment variable `PYTHONPATH`
3. the folders of the Python installation, where installed packages live (`site-packages`)

Our package will live in `src/heron/`, and `src/` is in none of these places. So, until `heron` is **installed**, we run it with:

```
PYTHONPATH=src python3 -m heron data/samples
```

You can also change `sys.path` from code (`sys.path.append(...)`), but avoid it: it hides how the program finds its modules. `dir(module)` lists the names a module defines, and `help(module)` prints its docstrings.

Step 1: a package under src/ (1/2)

Move `triage.py` into a new package folder, with `git mv`, so that git keeps its history. Then create the two special files:

```
mkdir -p src/heron
git mv triage.py src/heron/cli.py
```

`src/heron/__init__.py` holds the docstring of the package and its version; `src/heron/__main__.py` runs the command line:

```
"""Phishing email triage: indicators, scoring, verdicts."""

__version__ = "0.1.0"
```

```
"""Allow ``python -m heron <folder>``."""

from heron.cli import main

raise SystemExit(main())
```

Step 1: a package under src/ (2/2)

At the end of `cli.py`, the `if __name__ == "__main__":` block becomes a function `main()` that **returns** the exit code. `__main__.py` passes it to `SystemExit`, which ends the program with that code:

```
-if __name__ == "__main__":
-    if len(sys.argv) != 2:
-        print("usage: python triage.py <folder-with-eml-files>", file=sys.stderr)
-        sys.exit(2)
+def main(argv: list[str] | None = None) -> int:
+    """Run the triage on the folder given on the command line; return the exit code."""
+    args = sys.argv[1:] if argv is None else argv
+    if len(args) != 1:
+        print("usage: python -m heron <folder-with-eml-files>", file=sys.stderr)
+        return 2
     try:
-        report(check_mail(sys.argv[1]))
+        report(check_mail(args[0]))
     except FileNotFoundError:
-        print("error: no such folder:", sys.argv[1], file=sys.stderr)
-        sys.exit(2)
+        print("error: no such folder:", args[0], file=sys.stderr)
+        return 2
+    return 0
```

A function is easier to reuse and test than a script block: a test can call `main(["data/samples"])` and check the returned code. The verdicts do not move.

Type hints

`main()` is the first function of `heron` with **type hints**: after each parameter, `: type` says what kind of value it expects; after the parentheses, `-> type` says what the function returns.

```
def main(argv: list[str] | None = None) -> int:
```

`argv` is a list of strings, or `None` (no value: then `main()` reads `sys.argv`); the function returns an `int`, the exit code.

- Python **ignores** type hints when it runs the code: a wrong hint does not raise an error
- they are **documentation that tools can read**: the reader sees what goes in and what comes out without reading the body, and the IDE completes names and marks some mistakes as you type
- a **type checker** reads them and finds errors without running the code: a later lecture of this module adds one, mypy, to `heron`

Our rule from now on: every function whose signature (name, parameters, result) we write or change gets type hints. Code that we only move keeps its old form until we change it, so for a few steps the code is partly typed; at the end of today every function has hints.

Why src/? The src layout

`heron` puts its package in `src/heron/`, not in `heron/` at the repository root. This is the **src layout**.

- with a package at the root, `import heron` works from the root folder by accident (the current folder is on `sys.path`), so you can run code that would fail for a user who installed the package
- with `src/`, `import heron` works only through `PYTHONPATH` or a real installation, so you always run the code the way a user gets it
- tests (Module 4) and docs live **outside** the package: `tests/`, `docs/`

```
PYTHONPATH=src python3 -m heron data/samples | sort | diff /tmp/before.txt -   # no output = same verdicts
```

The interface of a Python module: the underscore

A name that starts with an underscore (_) is **internal**: a secret of its module. Other modules should not use it. At the end of today, `parser.py` contains:

```
_URL_RE = re.compile(...)                                  # internal constant
def _domain(header_value: str) -> str | None: ...          # internal helper
def parse_eml(raw: str, email_id: str) -> ParsedEmail: ...   # the interface
```

The interface of `parser.py` is one function, `parse_eml()`. Python does **not** enforce this: `parser._domain("...")` works, but it is strongly discouraged. The author may change or delete a _ name at any time, and your code would break.

Step 2: one module per responsibility

`cli.py` still holds all the code. Cut and paste each part into its own module (no line is rewritten):

```
src/heron/
├── __init__.py     # the package: its docstring and __version__
├── __main__.py     # "python -m heron" runs cli.main()
├── parser.py       # sender_domain(): read an address
├── features.py     # the weights, BRANDS, KEYWORDS and the five *_score() functions
├── scorer.py       # the thresholds, score_email() and verdict_for()
└── cli.py          # check_mail(), report() and main(): files in, report out
```

Each module gets a docstring and imports only what it needs. The two regular expressions become `_URL_RE` and `_IP_URL_RE`: they are secrets of `features.py`. Only 11 lines are new; the rest is moved.

Step 2: dependencies go one way

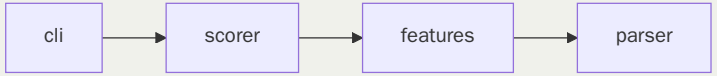
`scorer.py` after step 2 imports the indicators from `features.py`:

```
"""Combine the indicator points into a score and a verdict."""

from email import message_from_string, policy

from heron.features import auth_score, brand_score, keyword_score, link_score, reply_to_score

PHISHING_THRESHOLD = 5
SUSPICIOUS_THRESHOLD = 3
```



The arrows go in one direction only: there is no cycle. So `parser` can be understood and tested alone, then `features`, then `scorer`, and `cli` last.

Interfaces in heron

Step 3: a data class for a parsed message

The checks need a few facts about a message: who sent it, its domains, the authentication results, its text and its links. A new module, `models.py`, describes them in one type:

```python
from dataclasses import dataclass


@dataclass(frozen=True)
class ParsedEmail:
    """The parts of one message that the indicator checks read."""

    email_id: str
    sender: str
    from_domain: str | None
    reply_to_domain: str | None
    auth_results: str
    text: str
    links: tuple[str, ...]
```

`@dataclass` writes the boring methods for you: `__init__`, a readable `__repr__` and `==`. With `frozen=True` an object cannot change after it is created: `email.sender = "x"` raises `FrozenInstanceError`. Each field has a **type annotation**; `str | None` means "a string, or no value". Nothing uses the class yet.

Step 4: read each message once, with parse_eml()

`parser.py` gets the function that turns a raw message into a `ParsedEmail`. It reads the message with the `email` library **once**:

```
def parse_eml(raw: str, email_id: str) -> ParsedEmail:
    """Read one raw message once and keep only what the checks need."""
    msg = message_from_string(raw, policy=policy.default)
    sender = str(msg.get("From", ""))
    return ParsedEmail(
        email_id=email_id,
        sender=sender,
        from_domain=sender_domain(sender),
        reply_to_domain=sender_domain(str(msg.get("Reply-To", ""))),
        auth_results=str(msg.get("Authentication-Results", "")).lower(),
        text=raw.lower(),
        links=tuple(_URL_RE.findall(raw)),
    )
```

The URL pattern moves here as `_URL_RE`, and `sender_domain()` gets type annotations. `parse_eml()` is the **isolation layer**: the only code that talks to the `email` library. Nothing calls it yet.

Step 5: score a ParsedEmail, hide the details (1/2)

Now `cli.py` calls `score_email(parse_eml(raw, fn))`, and `score_email()` passes the whole `ParsedEmail` to each indicator:

```
-def score_email(raw):
-    """Sum the points of every indicator for one raw message."""
-    msg = message_from_string(raw, policy=policy.default)
-    frm = str(msg.get("From", ""))
-    reply_to = str(msg.get("Reply-To", ""))
-    return (keyword_score(raw) + link_score(raw) + brand_score(frm)
-            + auth_score(raw) + reply_to_score(frm, reply_to))
+def score_email(email: ParsedEmail) -> int:
+    """Sum the points of every indicator for one parsed message."""
+    return (keyword_score(email) + link_score(email) + brand_score(email)
+            + auth_score(email) + reply_to_score(email))
```

This is **information hiding**: only `parser.py` imports the `email` library now, and `sender_domain()` becomes its secret `_domain()`. The commit touches four files: `cli.py`, `parser.py`, `features.py` and `scorer.py`.

Step 5: score a ParsedEmail, hide the details (2/2)

In `features.py`, each indicator takes the `ParsedEmail` and reads the field it needs, instead of the raw text. Two of the five:

```python
def keyword_score(email: ParsedEmail) -> int:
    """One point for each pressure keyword found anywhere in the message."""
    return sum(W_KEYWORD for kw in KEYWORDS if kw in email.text)


def auth_score(email: ParsedEmail) -> int:
    """Points when SPF or DMARC failed."""
    if "spf=fail" in email.auth_results or "dmarc=fail" in email.auth_results:
        return W_AUTH_FAIL
    return 0
```

Now all five indicators have the same **signature** (the types of their parameters and of their result): they take a `ParsedEmail` and return an `int`, the points they add. We call such a function a **check**. Their signatures change, so they get type hints. In `reply_to_score()`, both domains are `str | None`: `if from_domain and reply_domain` makes sure that both are strings before they are compared (we'll come back to this with the type checker).

Step 6: one list of checks

`score_email()` still names the five checks one by one, so a new indicator means editing it. Register the checks in one tuple at the end of `features.py`, and loop over it in `scorer.py`:

```
ALL_CHECKS: tuple[Callable[[ParsedEmail], int], ...] = (
    keyword_score,
    link_score,
    brand_score,
    auth_score,
    reply_to_score,
)
```

```
-    return (keyword_score(email) + link_score(email) + brand_score(email)
-            + auth_score(email) + reply_to_score(email))
+    return sum(check(email) for check in ALL_CHECKS)
```

The type says: a tuple of any length whose items are functions from `ParsedEmail` to `int` (`Callable` comes from `collections.abc`). `scorer.py` imports only `ALL_CHECKS`: it no longer names any single check.

Step 7: one entry point, triage()

`score_email()` becomes `triage()`: it returns a second data class, `TriageResult`, with the score and the verdict of one message:

```
def triage(email: ParsedEmail) -> TriageResult:
    """Run every check on one message and decide its verdict."""
    score = sum(check(email) for check in ALL_CHECKS)
    return TriageResult(email.email_id, score, verdict_for(score))
```

`TriageResult` has three fields: `email_id`, `score` and `verdict`. `cli.py` now calls only `triage(parse_eml(raw, fn))` and prints the same lines as before. One simple function in front of the whole engine is called a **facade**.

Now the behaviour changes

From refactor to feat

Steps 1–7 changed the structure and kept every verdict and score: they are **refactorings**. The next steps change what `heron` prints or decides: they are **features**, and their commit messages start with `feat:`.

- after a `feat:` step the safety net prints differences, and that is expected: each slide says what changes
- read the differences, check that they are the announced ones, and only then save the new output as the reference for the next step

```
PYTHONPATH=src python3 -m heron data/samples | sort | diff /tmp/before.txt -   # the expected changes
PYTHONPATH=src python3 -m heron data/samples | sort > /tmp/before.txt          # the new reference
```

Keeping refactorings and features in separate commits lets a reviewer know which commits may change results.

Step 8: named verdicts with StrEnum

The verdicts are still the strings `"ok"`, `"suspicious"` and `"PHISHING"`: a typo such as `"phishng"` would go unnoticed. An **enumeration** lists the allowed values once, with a name each. In `models.py`:

```python
from enum import StrEnum


class Verdict(StrEnum):
    """The outcome of the triage, from harmless to dangerous."""

    BENIGN = "benign"
    SUSPICIOUS = "suspicious"
    PHISHING = "phishing"
```

`verdict_for()` returns `Verdict.PHISHING`, `Verdict.SUSPICIOUS` or `Verdict.BENIGN`, `TriageResult.verdict` gets the type `Verdict`, and the `flagged` line in `cli.py` compares with `Verdict.PHISHING` (otherwise it stays empty). `StrEnum` (Python 3.11 or newer) is a subclass of `str`: each member **is** a string, so it prints and compares like one. The output changes only in the labels: `ok` → `benign`, `PHISHING` → `phishing`.

Step 9: one line per email

`check_mail()` now returns the `TriageResult` objects, in name order, and `report()` prints one line for each:

```
def report(results: list[TriageResult]) -> None:
    """A line for each email, with its verdict and its score."""
    for r in results:
        print(f"{r.email_id:<28} {r.verdict:<10} score={r.score}")
```

`{r.email_id:<28}` pads the name to 28 characters, so the columns line up. The "checked", "flagged" and `results.txt` lines go away: the report is printed only.

Step 10: the command line with argparse

`main()` reads `sys.argv` by hand. The standard library module **`argparse`** does it for us: we declare the arguments, and it writes the usage line, the error messages and a `--help` option:

```
def build_parser() -> argparse.ArgumentParser:
    """Return the parser of the command-line arguments."""
    parser = argparse.ArgumentParser(prog="heron", description="Triage phishing emails.")
    parser.add_argument("folder", help="a folder of .eml files")
    return parser
```

In `main()`, the four lines that checked `sys.argv` become `args = build_parser().parse_args(argv)`, and `args[0]` becomes `args.folder`. Without an argument, `argparse` prints `heron: error: the following arguments are required: folder` and exits with code 2, as before.

Step 11: one file or a folder, and --json

The argument becomes a **path** (`pathlib.Path`): one `.eml` file or a folder. A second option, `--json`, prints the results for other programs:

```
-    parser.add_argument("folder", help="a folder of .eml files")
+    parser.add_argument("path", type=Path, help="an .eml file or a folder of .eml files")
+    parser.add_argument("--json", action="store_true", help="machine-readable output")
...
+    files = sorted(path.glob("*.eml")) if path.is_dir() else [path]
...
+    if args.json:
+        print(json.dumps([asdict(r) for r in results], indent=2))
+    else:
+        report(results)
```

`asdict()` (from `dataclasses`) turns a `TriageResult` into a dictionary, and `json.dumps()` turns it into text. The other edits (the imports of `json`, `asdict` and `Path`, and the new name `path` in `check_mail()` and `main()`) follow from these lines. Try `PYTHONPATH=src python3 -m heron data/samples/prize-scam.eml --json`. This step also sets `__version__ = "0.2.0"`: while `heron` is at version 0.x, its middle number goes up once per module stage (we'll talk about version numbers soon).

Step 12: update the README

The README still says `python triage.py data/samples`. Update the usage section **in your own words**: the new command, the single-file path, `--json` and `--help`. Then delete `notes.txt`: the README now says how to run `heron`, and the thresholds are in the code.

```
git rm notes.txt
git add README.md
git commit -m "docs: update the README for the package layout"
git push
```

The prefix `docs:` marks a commit that changes only documentation.

Where we are

Your `lab-01-start` now has the same code as my `lab-01-solution`:

```
ls src/heron     # __init__.py __main__.py cli.py features.py models.py parser.py scorer.py
PYTHONPATH=src python3 -m heron data/samples/prize-scam.eml
# prize-scam.eml                phishing   score=9
```

Module	Responsibility	Interface
`models`	the data types	`ParsedEmail`, `Verdict`, `TriageResult`
`parser`	raw text to `ParsedEmail`	`parse_eml()`
`features`	the indicators	five `*_score` checks, `ALL_CHECKS`
`scorer`	points to score and verdict	`triage()`, `verdict_for()`
`cli`	files in, report out	`main()`

The modules `models`, `parser`, `features` and `scorer` do no input or output: they are the **domain** of `heron`. Only `cli` touches files and the screen.

SOLID

SOLID: five design principles

SOLID is a set of five principles (collected by Robert C. Martin) that keep modules cohesive and loosely coupled:

- **S**ingle responsibility
- **O**pen–closed
- **L**iskov substitution
- **I**nterface segregation
- **D**ependency inversion

They were written for classes, but they apply to modules and functions too. The next slides read each principle on the code you wrote today.

S: single responsibility

A module (or class, or function) has one reason to change.

- `parser.py` changes only if the way we read emails changes
- `scorer.py` changes only if the way we combine the points changes
- `cli.py` changes only if the command line changes

In `triage.py`, any change (a new indicator, a new output format, a new input folder) meant editing the same file. Step 2 gave each responsibility its own module.

O: open–closed

Code should be open to extension, but closed to modification: you add behaviour by adding code, not by editing code that already works.

- in `heron`, every check has the same signature (step 5), and all checks are registered in `ALL_CHECKS` (step 6)
- `triage()` loops over `ALL_CHECKS` and never names a single check
- so a new indicator is one new function plus one line in `ALL_CHECKS`: the code of `triage()` stays closed, the list of checks stays open

L: Liskov substitution

An object of a subclass must work wherever an object of its parent class is expected, with no surprises: same methods, same promises.

`Verdict` (step 8) is a subclass of `str`, so a `Verdict` works wherever a string is expected:

```
>>> from heron.models import Verdict
>>> v = Verdict.PHISHING
>>> isinstance(v, str), v == "phishing", v.upper()
(True, True, 'PHISHING')
>>> f"[{v:<10}]"
'[phishing  ]'
```

This is why steps 8–11 needed no special code for the verdict: the report pads it with `:<10` like any string, and `json.dumps()` writes it as `"phishing"`. A subclass that broke one of these promises (for example, an `upper()` that returned something else) would break every client.

I: interface segregation

A client should not depend on methods it does not use. Many small interfaces are better than one large one.

- the checks do not receive the standard library's `email.message.EmailMessage`, a class with dozens of methods
- they receive `ParsedEmail` (step 3): a small data class with only the seven fields that triage needs
- so the checks do not depend on the `email` library at all: only `parser.py` does (step 5), and it is the **isolation layer**

D: dependency inversion

High-level code should not depend on low-level details: both should depend on an abstraction. The high-level module receives the low-level one from outside, through an interface that it defines.

- `heron` does **not** apply this principle yet. `scorer.py` imports `ALL_CHECKS` from the concrete module `heron.features`: that is the open–closed principle, not dependency inversion
- what `heron` already has is the right direction between layers: the domain does not import the command line; the command line imports the domain
- Module 6 applies dependency inversion: the web API defines a **port** (an interface) for storage, and the concrete storage is passed in from outside

Where we are

Look at your commit history:

```
git log --oneline
git diff --stat <stuff-id>                              # <stuff-id>: the ID of "stuff", the last line of git log
```

What is still missing:

- `heron` cannot be **installed**, and it needs `PYTHONPATH=src`: we'll add `pyproject.toml` and `uv`
- nothing **checks** style, types or documentation automatically
- nothing **proves** it works: no tests

Takeaways

- a **module** can be understood, changed and tested separately; modularity manages complexity, change, teamwork and reuse
- aim for **high cohesion** inside modules and **low coupling** between them
- a module offers an **interface** and hides its implementation (information hiding); in Python the `_` prefix marks the secrets, by convention
- Python: a module is a file, a package is a folder (`__init__.py`, `__main__.py`); `__name__ == "__main__"` separates running from importing; `sys.path` decides where imports are found; the **src layout** makes you run the code as a user gets it
- frozen **data classes** describe the data (`ParsedEmail`, `TriageResult`); a **`StrEnum`** names the allowed values (`Verdict`)
- `refactor:` commits keep the output, `feat:` commits change it on purpose; keep them apart
- **SOLID**: single responsibility, open–closed (`ALL_CHECKS`), Liskov (`Verdict` is a `str`), interface segregation (`ParsedEmail`); dependency inversion comes in Module 6
- next: make `heron` installable, with declared dependencies

Appendix: every step in full

How to read the appendix

The slides of the lecture show only the key lines of each step. The next slides show the **complete change** of every step, as `git diff` prints it, so that you can check your code line by line, or type a step you missed.

- a line that starts with `-` is removed, a line that starts with `+` is added, a line that starts with a space is unchanged and only shows where the change goes
- `@@ -12,3 +12,3 @@` gives the position of the lines that follow: line 12 of the old file (3 lines shown) becomes line 12 of the new file
- a **new file** is shown in full, as plain code, and `...` stands for lines left out, as the slide says; the slide also says whether you **write it from scratch** or **copy** its code from another file (cut and paste, then the few changes it lists)
- when a step needs more than one slide, the change continues on the next slide, in order

Step 1 in full (1/2)

Commit `refactor: move triage.py into the heron package under src/`: the complete change, file by file. First `mkdir -p src/heron` and `git mv triage.py src/heron/cli.py` (slide *Step 1: a package under src/ (1/2)*).

New file `src/heron/__init__.py`, written from scratch:

```
"""Phishing email triage: indicators, scoring, verdicts."""

__version__ = "0.1.0"
```

New file `src/heron/__main__.py`, written from scratch:

```
"""Allow ``python -m heron <folder>``."""

from heron.cli import main

raise SystemExit(main())
```

Step 1 in full (2/2)

Step 1, continued. `triage.py` → `src/heron/cli.py`, moved with `git mv` (not a new file: git keeps its history), then changed at the end:

```
@@ -116,10 +116,13 @@

-if __name__ == "__main__":
-    if len(sys.argv) != 2:
-        print("usage: python triage.py <folder-with-eml-files>", file=sys.stderr)
-        sys.exit(2)
+def main(argv: list[str] | None = None) -> int:
+    """Run the triage on the folder given on the command line; return the exit code."""
+    args = sys.argv[1:] if argv is None else argv
+    if len(args) != 1:
+        print("usage: python -m heron <folder-with-eml-files>", file=sys.stderr)
+        return 2
     try:
-        report(check_mail(sys.argv[1]))
+        report(check_mail(args[0]))
     except FileNotFoundError:
-        print("error: no such folder:", sys.argv[1], file=sys.stderr)
-        sys.exit(2)
+        print("error: no such folder:", args[0], file=sys.stderr)
+        return 2
+    return 0
```

Step 2 in full (1/6)

Commit `refactor: split the package into one module per responsibility`: the complete change. Almost every line is **moved**, not rewritten: `cli.py` keeps only `check_mail()`, `report()` and `main()`, and everything above them goes to three new files.

`src/heron/cli.py`, the top of the file (`...` stands for the deleted lines from `# Points added by each indicator` to the end of `verdict_for()`):

```
@@ -1,92 +1,7 @@
-# quick phishing checker for the mail export - L. Garcia, march 2025
+"""Command-line interface: ``python -m heron <folder>``."""
+
 import os
-import re
 import sys
-from email import message_from_string, policy
-from email.utils import parseaddr
...
-    return "ok"
+from heron.scorer import score_email, verdict_for
```

Step 2 in full (2/6)

Step 2, continued. New file `src/heron/parser.py`. Copy `sender_domain()` from `cli.py` (cut and paste, unchanged); write the module docstring and the import:

```
"""Read the triage-relevant parts of a raw ``.eml`` message."""

from email.utils import parseaddr


def sender_domain(header_value):
    """Return the domain of the address in a From/Reply-To value, or None."""
    _, address = parseaddr(header_value)
    if "@" not in address:
        return None
    return address.rsplit("@", 1)[1].lower()
```

Step 2 in full (3/6)

Step 2, continued. New file `src/heron/scorer.py`. Copy the two thresholds, `score_email()` and `verdict_for()` from `cli.py` (unchanged); write the module docstring and the imports:

```
"""Combine the indicator points into a score and a verdict."""

from email import message_from_string, policy

from heron.features import auth_score, brand_score, keyword_score, link_score, reply_to_score

PHISHING_THRESHOLD = 5
SUSPICIOUS_THRESHOLD = 3


def score_email(raw):
    """Sum the points of every indicator for one raw message."""
    msg = message_from_string(raw, policy=policy.default)
    frm = str(msg.get("From", ""))
    reply_to = str(msg.get("Reply-To", ""))
    return (keyword_score(raw) + link_score(raw) + brand_score(frm)
            + auth_score(raw) + reply_to_score(frm, reply_to))


def verdict_for(score):
    """Map a score to a verdict."""
    if score >= PHISHING_THRESHOLD:
        return "PHISHING"
    if score >= SUSPICIOUS_THRESHOLD:
        return "suspicious"
    return "ok"
```

Step 2 in full (4/6)

Step 2, continued. New file `src/heron/features.py`. Copy from `cli.py` the weights (without the two thresholds, which go to `scorer.py`), the lists, the two regular expressions and the five indicators; then add a leading `_` to the names of the two regular expressions, and write the module docstring and the imports.

```
"""Phishing indicators: each function returns the points one indicator adds."""

import re

from heron.parser import sender_domain

W_KEYWORD = 1
W_IP_LINK = 3
W_PUNYCODE_LINK = 3
W_BRAND_MISMATCH = 3
W_AUTH_FAIL = 2
W_REPLY_TO_MISMATCH = 2

BRANDS = ["paypal", "microsoft", "amazon"]

KEYWORDS = ["urgent", "verify", "suspended", "password", "expires", "act now",
            "congratulations", "winner", "claim", "immediately", "gift card"]

_URL_RE = re.compile("https?://[^\\s\"'<>]+")
_IP_URL_RE = re.compile("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+")


def keyword_score(raw):
    """One point for each pressure keyword found anywhere in the message."""
    low = raw.lower()
    return sum(W_KEYWORD for kw in KEYWORDS if kw in low)
```

Step 2 in full (5/6)

Step 2, continued: `src/heron/features.py`, continued.

```
def link_score(raw):
    """Points for links to a bare IP address or to a punycode domain."""
    s = 0
    for url in _URL_RE.findall(raw):
        if _IP_URL_RE.match(url):
            s = s + W_IP_LINK
        if "xn--" in url:
            s = s + W_PUNYCODE_LINK
    return s


def brand_score(frm):
    """Points when the sender name claims a brand but the address is elsewhere."""
    low = frm.lower()
    return sum(W_BRAND_MISMATCH for brand in BRANDS
               if brand in low and brand + ".com" not in low)
```

Step 2 in full (6/6)

Step 2, continued: the end of `src/heron/features.py`.

```
def auth_score(raw):
    """Points when SPF or DMARC failed."""
    low = raw.lower()
    if "spf=fail" in low or "dmarc=fail" in low:
        return W_AUTH_FAIL
    return 0


def reply_to_score(frm, reply_to):
    """Points when replies go to a different domain than the sender's."""
    from_domain = sender_domain(frm)
    reply_domain = sender_domain(reply_to)
    if from_domain and reply_domain and from_domain != reply_domain:
        return W_REPLY_TO_MISMATCH
    return 0
```

Step 3 in full

Commit `refactor: describe a parsed message with a data class`: the complete change.

New file `src/heron/models.py`, written from scratch:

```
"""The data types of the triage domain."""

from dataclasses import dataclass


@dataclass(frozen=True)
class ParsedEmail:
    """The parts of one message that the indicator checks read."""

    email_id: str
    sender: str
    from_domain: str | None
    reply_to_domain: str | None
    auth_results: str
    text: str
    links: tuple[str, ...]
```

Step 4 in full (1/2)

Commit `refactor: read each message once with parse_eml()`: the complete change.

`src/heron/parser.py`, the top of the file:

```
@@ -2,6 +2,12 @@

+import re
+from email import message_from_string, policy
 from email.utils import parseaddr

+from heron.models import ParsedEmail

-def sender_domain(header_value):
+_URL_RE = re.compile("https?://[^\\s\"'<>]+")
+
+
+def sender_domain(header_value: str) -> str | None:
     """Return the domain of the address in a From/Reply-To value, or None."""
```

Step 4 in full (2/2)

Step 4, continued: `parse_eml()` at the end of `src/heron/parser.py`.

```
@@ -11 +17,16 @@
      return address.rsplit("@", 1)[1].lower()
+
+
+def parse_eml(raw: str, email_id: str) -> ParsedEmail:
+    """Read one raw message once and keep only what the checks need."""
+    msg = message_from_string(raw, policy=policy.default)
+    sender = str(msg.get("From", ""))
+    return ParsedEmail(
+        email_id=email_id,
+        sender=sender,
+        from_domain=sender_domain(sender),
+        reply_to_domain=sender_domain(str(msg.get("Reply-To", ""))),
+        auth_results=str(msg.get("Authentication-Results", "")).lower(),
+        text=raw.lower(),
+        links=tuple(_URL_RE.findall(raw)),
+    )
```

Step 5 in full (1/5)

Commit `refactor: score a ParsedEmail instead of the raw text`: the complete change, file by file.

`src/heron/cli.py`:

```
@@ -5,2 +5,3 @@

+from heron.parser import parse_eml
 from heron.scorer import score_email, verdict_for
@@ -16,3 +17,3 @@
             raw = f.read()
-        s = score_email(raw)
+        s = score_email(parse_eml(raw, fn))
         verdicts.append((fn, verdict_for(s), s))
```

`src/heron/features.py`, the top of the file:

```
@@ -4,3 +4,3 @@

-from heron.parser import sender_domain
+from heron.models import ParsedEmail

@@ -18,3 +18,2 @@

-_URL_RE = re.compile("https?://[^\\s\"'<>]+")
 _IP_URL_RE = re.compile("https?://[0-9]+\\.[0-9]+\\.[0-9]+\\.[0-9]+")
```

Step 5 in full (2/5)

Step 5, continued: `src/heron/features.py`, the first three indicators.

```
@@ -22,12 +21,11 @@

-def keyword_score(raw):
+def keyword_score(email: ParsedEmail) -> int:
     """One point for each pressure keyword found anywhere in the message."""
-    low = raw.lower()
-    return sum(W_KEYWORD for kw in KEYWORDS if kw in low)
+    return sum(W_KEYWORD for kw in KEYWORDS if kw in email.text)


-def link_score(raw):
+def link_score(email: ParsedEmail) -> int:
     """Points for links to a bare IP address or to a punycode domain."""
     s = 0
-    for url in _URL_RE.findall(raw):
+    for url in email.links:
         if _IP_URL_RE.match(url):
@@ -39,5 +37,5 @@

-def brand_score(frm):
+def brand_score(email: ParsedEmail) -> int:
     """Points when the sender name claims a brand but the address is elsewhere."""
-    low = frm.lower()
+    low = email.sender.lower()
     return sum(W_BRAND_MISMATCH for brand in BRANDS
```

Step 5 in full (3/5)

Step 5, continued: `src/heron/features.py`, the last two indicators.

```
@@ -46,6 +44,5 @@

-def auth_score(raw):
+def auth_score(email: ParsedEmail) -> int:
     """Points when SPF or DMARC failed."""
-    low = raw.lower()
-    if "spf=fail" in low or "dmarc=fail" in low:
+    if "spf=fail" in email.auth_results or "dmarc=fail" in email.auth_results:
         return W_AUTH_FAIL
@@ -54,6 +51,5 @@

-def reply_to_score(frm, reply_to):
+def reply_to_score(email: ParsedEmail) -> int:
     """Points when replies go to a different domain than the sender's."""
-    from_domain = sender_domain(frm)
-    reply_domain = sender_domain(reply_to)
+    from_domain, reply_domain = email.from_domain, email.reply_to_domain
     if from_domain and reply_domain and from_domain != reply_domain:
```

Step 5 in full (4/5)

Step 5, continued.

`src/heron/parser.py`:

```
@@ -11,3 +11,3 @@

-def sender_domain(header_value: str) -> str | None:
+def _domain(header_value: str) -> str | None:
     """Return the domain of the address in a From/Reply-To value, or None."""
@@ -26,4 +26,4 @@
         sender=sender,
-        from_domain=sender_domain(sender),
-        reply_to_domain=sender_domain(str(msg.get("Reply-To", ""))),
+        from_domain=_domain(sender),
+        reply_to_domain=_domain(str(msg.get("Reply-To", ""))),
         auth_results=str(msg.get("Authentication-Results", "")).lower(),
```

Step 5 in full (5/5)

Step 5, continued.

`src/heron/scorer.py`:

```
@@ -2,5 +2,4 @@

-from email import message_from_string, policy
-
 from heron.features import auth_score, brand_score, keyword_score, link_score, reply_to_score
+from heron.models import ParsedEmail

@@ -10,9 +9,6 @@

-def score_email(raw):
-    """Sum the points of every indicator for one raw message."""
-    msg = message_from_string(raw, policy=policy.default)
-    frm = str(msg.get("From", ""))
-    reply_to = str(msg.get("Reply-To", ""))
-    return (keyword_score(raw) + link_score(raw) + brand_score(frm)
-            + auth_score(raw) + reply_to_score(frm, reply_to))
+def score_email(email: ParsedEmail) -> int:
+    """Sum the points of every indicator for one parsed message."""
+    return (keyword_score(email) + link_score(email) + brand_score(email)
+            + auth_score(email) + reply_to_score(email))
```

Step 6 in full (1/2)

Commit `refactor: run the checks from one list, ALL_CHECKS`: the complete change, file by file.

`src/heron/features.py`:

```
@@ -3,2 +3,3 @@
 import re
+from collections.abc import Callable

@@ -57 +58,10 @@
     return 0
+
+
+ALL_CHECKS: tuple[Callable[[ParsedEmail], int], ...] = (
+    keyword_score,
+    link_score,
+    brand_score,
+    auth_score,
+    reply_to_score,
+)
```

Step 6 in full (2/2)

Step 6, continued.

`src/heron/scorer.py`:

```
@@ -2,3 +2,3 @@

-from heron.features import auth_score, brand_score, keyword_score, link_score, reply_to_score
+from heron.features import ALL_CHECKS
 from heron.models import ParsedEmail
@@ -11,4 +11,3 @@
     """Sum the points of every indicator for one parsed message."""
-    return (keyword_score(email) + link_score(email) + brand_score(email)
-            + auth_score(email) + reply_to_score(email))
+    return sum(check(email) for check in ALL_CHECKS)
```

Step 7 in full (1/3)

Commit `refactor: triage() returns a TriageResult`: the complete change, file by file.

`src/heron/cli.py`:

```
@@ -6,3 +6,3 @@
 from heron.parser import parse_eml
-from heron.scorer import score_email, verdict_for
+from heron.scorer import triage

@@ -17,4 +17,4 @@
             raw = f.read()
-        s = score_email(parse_eml(raw, fn))
-        verdicts.append((fn, verdict_for(s), s))
+        result = triage(parse_eml(raw, fn))
+        verdicts.append((fn, result.verdict, result.score))
     return verdicts
```

Step 7 in full (2/3)

Step 7, continued.

`src/heron/models.py`:

```
@@ -16 +16,10 @@
     links: tuple[str, ...]
+
+
+@dataclass(frozen=True)
+class TriageResult:
+    """The verdict on one message, with its score."""
+
+    email_id: str
+    score: int
+    verdict: str
```

Step 7 in full (3/3)

Step 7, continued.

`src/heron/scorer.py`:

```
@@ -3,3 +3,3 @@
 from heron.features import ALL_CHECKS
-from heron.models import ParsedEmail
+from heron.models import ParsedEmail, TriageResult

@@ -9,8 +9,9 @@

-def score_email(email: ParsedEmail) -> int:
-    """Sum the points of every indicator for one parsed message."""
-    return sum(check(email) for check in ALL_CHECKS)
+def triage(email: ParsedEmail) -> TriageResult:
+    """Run every check on one message and decide its verdict."""
+    score = sum(check(email) for check in ALL_CHECKS)
+    return TriageResult(email.email_id, score, verdict_for(score))


-def verdict_for(score):
+def verdict_for(score: int) -> str:
     """Map a score to a verdict."""
```

Step 8 in full (1/3)

Commit `feat: name the verdicts benign, suspicious and phishing`: the complete change, file by file.

`src/heron/cli.py`:

```
@@ -5,2 +5,3 @@

+from heron.models import Verdict
 from heron.parser import parse_eml
@@ -29,3 +30,3 @@
         out.write(str(verdicts))
-    print("flagged:", [v[0] for v in verdicts if v[1] == "PHISHING"])
+    print("flagged:", [v[0] for v in verdicts if v[1] == Verdict.PHISHING])
```

Step 8 in full (2/3)

Step 8, continued.

`src/heron/models.py`:

```
@@ -3,2 +3,3 @@
 from dataclasses import dataclass
+from enum import StrEnum

@@ -18,2 +19,10 @@

+class Verdict(StrEnum):
+    """The outcome of the triage, from harmless to dangerous."""
+
+    BENIGN = "benign"
+    SUSPICIOUS = "suspicious"
+    PHISHING = "phishing"
+
+
 @dataclass(frozen=True)
@@ -24,2 +33,2 @@
     score: int
-    verdict: str
+    verdict: Verdict
```

Step 8 in full (3/3)

Step 8, continued.

`src/heron/scorer.py`:

```
@@ -3,3 +3,3 @@
 from heron.features import ALL_CHECKS
-from heron.models import ParsedEmail, TriageResult
+from heron.models import ParsedEmail, TriageResult, Verdict

@@ -15,8 +15,8 @@

-def verdict_for(score: int) -> str:
+def verdict_for(score: int) -> Verdict:
     """Map a score to a verdict."""
     if score >= PHISHING_THRESHOLD:
-        return "PHISHING"
+        return Verdict.PHISHING
     if score >= SUSPICIOUS_THRESHOLD:
-        return "suspicious"
-    return "ok"
+        return Verdict.SUSPICIOUS
+    return Verdict.BENIGN
```

Step 9 in full (1/2)

Commit `feat: print one line per email`: the complete change, in `src/heron/cli.py`. First the import and `check_mail()`:

```
@@ -5,3 +5,3 @@

-from heron.models import Verdict
+from heron.models import TriageResult
 from heron.parser import parse_eml
@@ -10,6 +10,6 @@

-def check_mail(folder):
-    """Score every .eml file in folder; return (file name, verdict, score) tuples."""
-    verdicts = []
-    for fn in os.listdir(folder):
+def check_mail(folder: str) -> list[TriageResult]:
+    """Triage every .eml file in folder, in name order."""
+    results = []
+    for fn in sorted(os.listdir(folder)):
         if not fn.endswith(".eml"):
```

Step 9 in full (2/2)

Step 9, continued: the end of `check_mail()` and the new `report()`, which replaces the old one.

```
@@ -18,15 +18,10 @@
                raw = f.read()
-           result = triage(parse_eml(raw, fn))
-           verdicts.append((fn, result.verdict, result.score))
-       return verdicts
-
-
-def report(verdicts):
-       """Print the verdicts and save them to results.txt."""
-       print("checked", len(verdicts), "mails")
-       for v in verdicts:
-           print("  ", v[0], "->", v[1], "(score", str(v[2]) + ")")
-       with open("results.txt", "w", encoding="utf-8") as out:
-           out.write(str(verdicts))
-       print("flagged:", [v[0] for v in verdicts if v[1] == Verdict.PHISHING])
+           results.append(triage(parse_eml(raw, fn)))
+       return results
+
+
+def report(results: list[TriageResult]) -> None:
+       """A line for each email, with its verdict and its score."""
+       for r in results:
+           print(f"{r.email_id:<28} {r.verdict:<10} score={r.score}")
```

Step 10 in full

Commit `feat: parse the command line with argparse`: the complete change.

`src/heron/cli.py`:

```
@@ -2,2 +2,3 @@

+import argparse
 import os
@@ -10,2 +11,9 @@

+def build_parser() -> argparse.ArgumentParser:
+    """Return the parser of the command-line arguments."""
+    parser = argparse.ArgumentParser(prog="heron", description="Triage phishing emails.")
+    parser.add_argument("folder", help="a folder of .eml files")
+    return parser
+
+
 def check_mail(folder: str) -> list[TriageResult]:
@@ -30,10 +38,7 @@
     """Run the triage on the folder given on the command line; return the exit code."""
-    args = sys.argv[1:] if argv is None else argv
-    if len(args) != 1:
-        print("usage: python -m heron <folder-with-eml-files>", file=sys.stderr)
-        return 2
+    args = build_parser().parse_args(argv)
     try:
-        report(check_mail(args[0]))
+        report(check_mail(args.folder))
     except FileNotFoundError:
-        print("error: no such folder:", args[0], file=sys.stderr)
+        print("error: no such folder:", args.folder, file=sys.stderr)
         return 2
```

Step 11 in full (1/3)

Commit `feat: triage a single .eml file, and add a --json option`: the complete change.

`src/heron/__init__.py`:

```
@@ -2,2 +2,2 @@

-__version__ = "0.1.0"
+__version__ = "0.2.0"
```

`src/heron/cli.py`:

```
@@ -1,6 +1,8 @@
-"""Command-line interface: ``python -m heron <folder>``."""
+"""Command-line interface: ``python -m heron <path>``."""

 import argparse
-import os
+import json
 import sys
+from dataclasses import asdict
+from pathlib import Path

@@ -14,3 +16,4 @@
     parser = argparse.ArgumentParser(prog="heron", description="Triage phishing emails.")
-    parser.add_argument("folder", help="a folder of .eml files")
+    parser.add_argument("path", type=Path, help="an .eml file or a folder of .eml files")
+    parser.add_argument("--json", action="store_true", help="machine-readable output")
     return parser
```

Step 11 in full (2/3)

Step 11, continued.

`src/heron/cli.py`, continued:

```
@@ -18,11 +21,10 @@

-def check_mail(folder: str) -> list[TriageResult]:
-    """Triage every .eml file in folder, in name order."""
+def check_mail(path: Path) -> list[TriageResult]:
+    """Triage one .eml file, or every .eml file in a folder, in name order."""
+    files = sorted(path.glob("*.eml")) if path.is_dir() else [path]
     results = []
-    for fn in sorted(os.listdir(folder)):
-        if not fn.endswith(".eml"):
-            continue
-        with open(os.path.join(folder, fn), encoding="utf-8", errors="ignore") as f:
+    for file in files:
+        with open(file, encoding="utf-8", errors="ignore") as f:
             raw = f.read()
-        results.append(triage(parse_eml(raw, fn)))
+        results.append(triage(parse_eml(raw, file.name)))
     return results
```

Step 11 in full (3/3)

Step 11, continued.

`src/heron/cli.py`, continued:

```
@@ -37,9 +39,13 @@
 def main(argv: list[str] | None = None) -> int:
-    """Run the triage on the folder given on the command line; return the exit code."""
+    """Run the triage on the path given on the command line; return the exit code."""
     args = build_parser().parse_args(argv)
     try:
-        report(check_mail(args.folder))
+        results = check_mail(args.path)
     except FileNotFoundError:
-        print("error: no such folder:", args.folder, file=sys.stderr)
+        print("error: no such file or folder:", args.path, file=sys.stderr)
         return 2
+    if args.json:
+        print(json.dumps([asdict(r) for r in results], indent=2))
+    else:
+        report(results)
     return 0
```

Step 12 in full

Commit `docs: update the README for the package layout`. The new text of `README.md` is yours (slide *Step 12: update the README*), so it is not shown here. The other half of the commit: `notes.txt` is deleted.

```
@@ -1,3 +0,0 @@
-run with: python triage.py <folder>    (e.g. data/samples)
-score >=5 means phishing, >=3 suspicious. keywords list is in the code.
-sample mails are in data/samples. output goes to results.txt
```